



Automated Maze Solver

CSE 316: Microcontroller and Microprocessor Sessional

Group members:

Irham Dipra (2205068)

Priyanjan Das Anabil (2205079)

Aroni Ananya (2205087)

Motivation

The project solves the challenge of an autonomous robot entering an unknown maze without simply following a pre-drawn line.

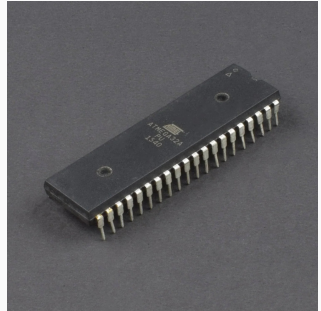
It demonstrates the physical application of embedded algorithms, spatial mapping, and graph traversal over basic reactive control.

This project leverages the group's strong software foundation in algorithms (like BFS and Flood Fill) and data structures, applying them to a complex physical constraint.

The robot must map the maze, remember visited cells, calculate the shortest exit path, and return to the start.

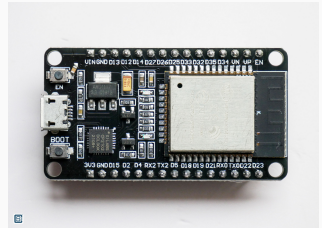
ATmega32 Dev Board:

Main low-level microcontroller for hardware interrupts and motor driving.



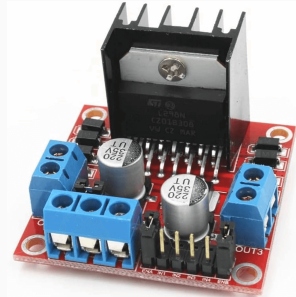
ESP32 DevKit V1:

Coprocessor responsible for processing sensor arrays and executing pathfinding algorithms.



L298N Motor Driver:

Controls the voltage and direction of the chassis motors based on microcontroller signals.



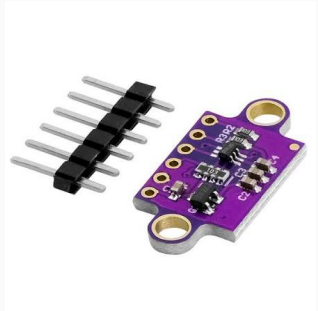
2WD Robot Chassis Kit:

Provides the physical movement platform, critically requiring included wheel encoder discs.



3x VL53L0X ToF Sensors:

High-precision time-of-flight sensors to detect walls on the front, left, and right sides.

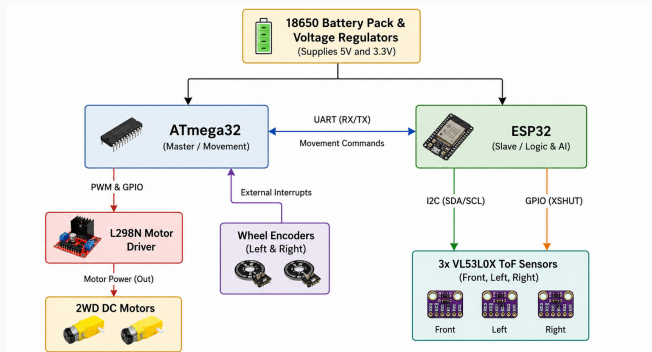


18650 Batteries & Regulators:

Provides reliable portable power for the motors and distinct logic levels for the microcontrollers.

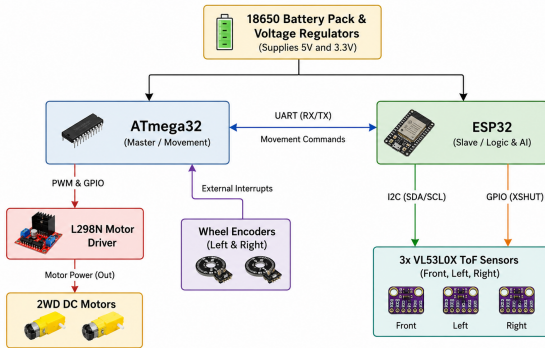


System Design



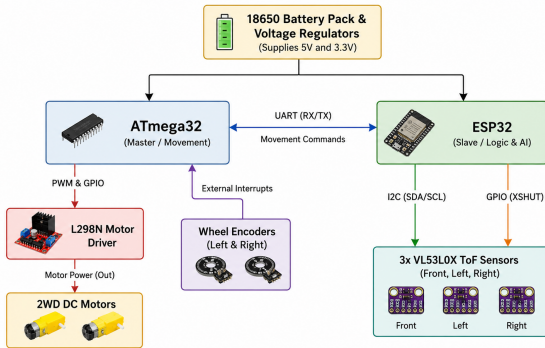
The system uses a master-slave configuration between the ATmega32 and the ESP32.

System Design



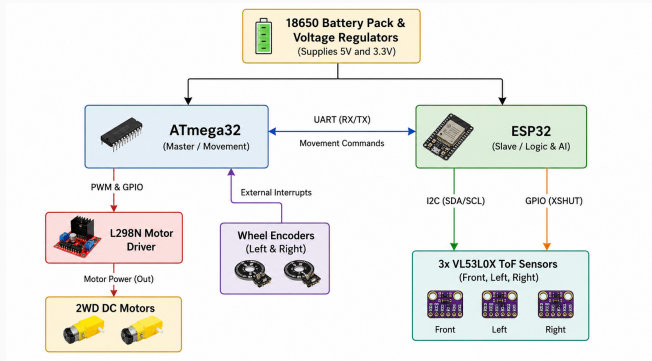
The ATmega32 is wired directly to the L298N motor driver and reads the wheel encoders.

System Design



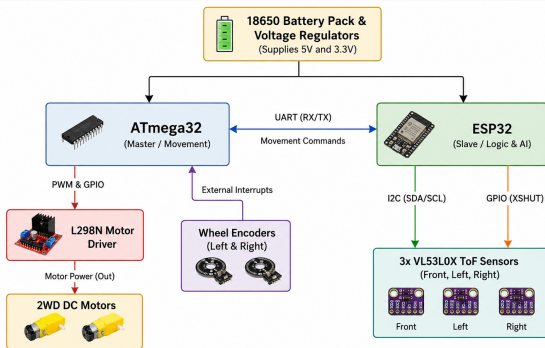
The ESP32 interfaces with the three VL53L0X Time-of-Flight sensors via the I2C bus.

System Design



The ESP32 calculates movement logic and sends high-level commands (e.g., "Move Forward 1 cell") to the ATmega32 over a UART serial connection.

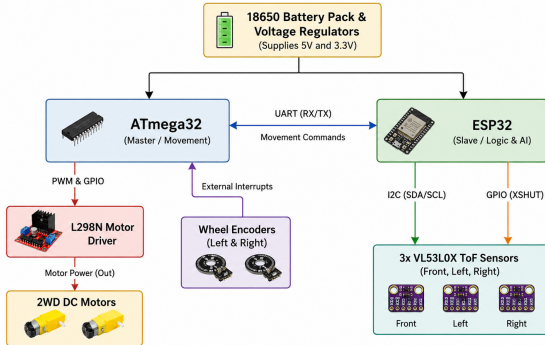
System Design



Important ATmega32 Interfaces Used:

UART (RX/TX Pins): Used to establish serial communication with the ESP32 to receive navigation commands.

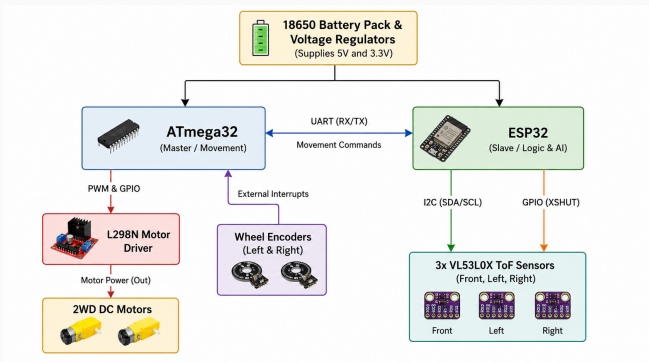
System Design



Important ATmega32 Interfaces Used:

External Interrupt Pins (INT0 / INT1): Used to read the rapid pulse signals generated by the left and right wheel encoders to track distance.

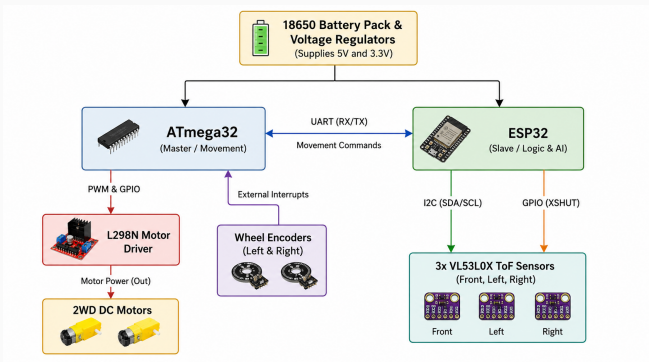
System Design



Important ATmega32 Interfaces Used:

PWM Channels: Used to send variable voltage signals to the L298N driver, allowing for dynamic motor speed control via a PID algorithm.

System Design



Important ATmega32 Interfaces Used:

Standard GPIO Pins: Used to send basic HIGH/LOW logic to the L298N driver to control the forward and reverse direction of the motors.

Role of the ATmega32

- **Primary Tasks:**

Role of the ATmega32

- **Primary Tasks:**
 - Dedicated real-time motion controller (translates ESP32 commands to motor timings).

Role of the ATmega32

- **Primary Tasks:**
 - Dedicated real-time motion controller (translates ESP32 commands to motor timings).
 - Executes continuous PID algorithm for straight driving and exact 90° turns.

Role of the ATmega32

- **Primary Tasks:**
 - Dedicated real-time motion controller (translates ESP32 commands to motor timings).
 - Executes continuous PID algorithm for straight driving and exact 90° turns.
- **Hardware Features Utilized:**

Role of the ATmega32

- **Primary Tasks:**

- Dedicated real-time motion controller (translates ESP32 commands to motor timings).
- Executes continuous PID algorithm for straight driving and exact 90° turns.

- **Hardware Features Utilized:**

- **INT0/INT1:** Read encoders for real-time distance tracking.

Role of the ATmega32

- **Primary Tasks:**

- Dedicated real-time motion controller (translates ESP32 commands to motor timings).
- Executes continuous PID algorithm for straight driving and exact 90° turns.

- **Hardware Features Utilized:**

- **INT0/INT1:** Read encoders for real-time distance tracking.
- **Timers & PWM:** Dynamic motor speed adjustments via PID.

Role of the ATmega32

- **Primary Tasks:**

- Dedicated real-time motion controller (translates ESP32 commands to motor timings).
- Executes continuous PID algorithm for straight driving and exact 90° turns.

- **Hardware Features Utilized:**

- **INT0/INT1:** Read encoders for real-time distance tracking.
- **Timers & PWM:** Dynamic motor speed adjustments via PID.
- **UART & GPIO:** Receive navigation commands and dictate motor directions.

Use of Other Controllers (ESP32)

- **Memory Constraint:** The ATmega32 only has 2KB of SRAM, which severely limits its ability to store complex maze arrays while handling heavy real-time motor control.

Use of Other Controllers (ESP32)

- **Memory Constraint:** The ATmega32 only has 2KB of SRAM, which severely limits its ability to store complex maze arrays while handling heavy real-time motor control.
- **Tasks:** The ESP32 handles the heavy lifting of the Flood Fill algorithm, tracks the state of the 256 cells, and manages the dynamic addressing sequence for the I2C sensors.

Use of Other Controllers (ESP32)

- **Memory Constraint:** The ATmega32 only has 2KB of SRAM, which severely limits its ability to store complex maze arrays while handling heavy real-time motor control.
- **Tasks:** The ESP32 handles the heavy lifting of the Flood Fill algorithm, tracks the state of the 256 cells, and manages the dynamic addressing sequence for the I2C sensors.
- **Communication:** It acts as the "brain," sending simplified directional commands to the ATmega32 over UART.

Implementation Plan: Expected Outcome

- **Expected Outcome:** A fully autonomous robot capable of navigating a 16×16 maze grid, mapping dead ends, and calculating the fastest route.

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**
 - Complete chassis assembly.

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**
 - Complete chassis assembly.
 - Establish PC simulation of the Flood Fill algorithm.

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**
 - Complete chassis assembly.
 - Establish PC simulation of the Flood Fill algorithm.
 - Finalize the ATmega32 PID controller so the robot can drive in a perfectly straight line for 1 meter.

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**
 - Complete chassis assembly.
 - Establish PC simulation of the Flood Fill algorithm.
 - Finalize the ATmega32 PID controller so the robot can drive in a perfectly straight line for 1 meter.
- **Project Update 2 (Weeks 3-5):**

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**
 - Complete chassis assembly.
 - Establish PC simulation of the Flood Fill algorithm.
 - Finalize the ATmega32 PID controller so the robot can drive in a perfectly straight line for 1 meter.
- **Project Update 2 (Weeks 3-5):**
 - Integrate the ToF sensors with the ESP32.

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**
 - Complete chassis assembly.
 - Establish PC simulation of the Flood Fill algorithm.
 - Finalize the ATmega32 PID controller so the robot can drive in a perfectly straight line for 1 meter.
- **Project Update 2 (Weeks 3-5):**
 - Integrate the ToF sensors with the ESP32.
 - Merge the mapping logic with the movement logic.

Implementation Plan: Project Updates

- **Project Update 1 (Weeks 1-2):**
 - Complete chassis assembly.
 - Establish PC simulation of the Flood Fill algorithm.
 - Finalize the ATmega32 PID controller so the robot can drive in a perfectly straight line for 1 meter.
- **Project Update 2 (Weeks 3-5):**
 - Integrate the ToF sensors with the ESP32.
 - Merge the mapping logic with the movement logic.
 - Finalize edge-case tuning for wall-drift correction.

System Testing Methodology

- **Phased Approach:** System tested in 4 phases to isolate hardware vs. algorithmic bugs.

System Testing Methodology

- **Phased Approach:** System tested in 4 phases to isolate hardware vs. algorithmic bugs.
- **Phase 1 (Locomotion & PID):** Test perfectly straight 1m driving and exact 90° turns using encoder counts.

System Testing Methodology

- **Phased Approach:** System tested in 4 phases to isolate hardware vs. algorithmic bugs.
- **Phase 1 (Locomotion & PID):** Test perfectly straight 1m driving and exact 90° turns using encoder counts.
- **Phase 2 (Sensor & I2C Calibration):** Verify XSHUT addressing to prevent I2C conflicts and calibrate ToF distances.

System Testing Methodology

- **Phased Approach:** System tested in 4 phases to isolate hardware vs. algorithmic bugs.
- **Phase 1 (Locomotion & PID):** Test perfectly straight 1m driving and exact 90° turns using encoder counts.
- **Phase 2 (Sensor & I2C Calibration):** Verify XSHUT addressing to prevent I2C conflicts and calibrate ToF distances.
- **Phase 3 (Algorithm Simulation):** Test Flood Fill algorithm on a PC simulation (virtual 16×16 grid) before flashing the ESP32.

System Testing Methodology

- **Phased Approach:** System tested in 4 phases to isolate hardware vs. algorithmic bugs.
- **Phase 1 (Locomotion & PID):** Test perfectly straight 1m driving and exact 90° turns using encoder counts.
- **Phase 2 (Sensor & I2C Calibration):** Verify XSHUT addressing to prevent I2C conflicts and calibrate ToF distances.
- **Phase 3 (Algorithm Simulation):** Test Flood Fill algorithm on a PC simulation (virtual 16×16 grid) before flashing the ESP32.
- **Phase 4 (Full Integration):** Test hardware/software synchronization in scaled-down physical mazes (e.g., 4×4 , 8×8).

Measurable Criteria for Project Success

- Metric 1: PID Synchronization Error (E_{PID})

Measurable Criteria for Project Success

- **Metric 1: PID Synchronization Error (E_{PID})**
 - Ensures motors spin at identical rates without drifting.

Measurable Criteria for Project Success

- **Metric 1: PID Synchronization Error (E_{PID})**
 - Ensures motors spin at identical rates without drifting.
 - Formula: $E_{PID} = |C_{left} - C_{right}|$ (where C = encoder pulse counts).

Measurable Criteria for Project Success

- **Metric 1: PID Synchronization Error (E_{PID})**
 - Ensures motors spin at identical rates without drifting.
 - Formula: $E_{PID} = |C_{left} - C_{right}|$ (where C = encoder pulse counts).
 - Success: Steady-state error must approach zero during a straight-line traversal.

Measurable Criteria for Project Success

- **Metric 1: PID Synchronization Error (E_{PID})**
 - Ensures motors spin at identical rates without drifting.
 - Formula: $E_{PID} = |C_{left} - C_{right}|$ (where C = encoder pulse counts).
 - Success: Steady-state error must approach zero during a straight-line traversal.
- **Metric 2: Algorithmic Path Efficiency (η_{path})**

Measurable Criteria for Project Success

- **Metric 1: PID Synchronization Error (E_{PID})**
 - Ensures motors spin at identical rates without drifting.
 - Formula: $E_{PID} = |C_{left} - C_{right}|$ (where C = encoder pulse counts).
 - Success: Steady-state error must approach zero during a straight-line traversal.
- **Metric 2: Algorithmic Path Efficiency (η_{path})**
 - Evaluates optimal path calculation after mapping.

Measurable Criteria for Project Success

- **Metric 1: PID Synchronization Error (E_{PID})**
 - Ensures motors spin at identical rates without drifting.
 - Formula: $E_{PID} = |C_{left} - C_{right}|$ (where C = encoder pulse counts).
 - Success: Steady-state error must approach zero during a straight-line traversal.
- **Metric 2: Algorithmic Path Efficiency (η_{path})**
 - Evaluates optimal path calculation after mapping.
 - Formula: $\eta_{path} = \frac{S_{optimal}}{S_{traversed}}$ (Optimal cells vs. actual cells traversed).

Measurable Criteria for Project Success

- **Metric 1: PID Synchronization Error (E_{PID})**
 - Ensures motors spin at identical rates without drifting.
 - Formula: $E_{PID} = |C_{left} - C_{right}|$ (where C = encoder pulse counts).
 - Success: Steady-state error must approach zero during a straight-line traversal.
- **Metric 2: Algorithmic Path Efficiency (η_{path})**
 - Evaluates optimal path calculation after mapping.
 - Formula: $\eta_{path} = \frac{S_{optimal}}{S_{traversed}}$ (Optimal cells vs. actual cells traversed).
 - Success: Achieving $\eta_{path} = 1$, proving Flood Fill found the absolute shortest path.

Separation of Features

- Essential Features (MVP):

Separation of Features

- **Essential Features (MVP):**
 - Maintain accurate digital (X, Y) coordinates and heading in memory.

Separation of Features

- **Essential Features (MVP):**
 - Maintain accurate digital (X, Y) coordinates and heading in memory.
 - Autonomously map the maze and find the center in a single pass.

Separation of Features

- **Essential Features (MVP):**
 - Maintain accurate digital (X , Y) coordinates and heading in memory.
 - Autonomously map the maze and find the center in a single pass.
 - Execute stable continuous PID motor control to navigate without crashing.

Separation of Features

- **Essential Features (MVP):**
 - Maintain accurate digital (X , Y) coordinates and heading in memory.
 - Autonomously map the maze and find the center in a single pass.
 - Execute stable continuous PID motor control to navigate without crashing.
- **Optional Features (Stretch Goals):**

Separation of Features

- **Essential Features (MVP):**
 - Maintain accurate digital (X , Y) coordinates and heading in memory.
 - Autonomously map the maze and find the center in a single pass.
 - Execute stable continuous PID motor control to navigate without crashing.
- **Optional Features (Stretch Goals):**
 - **The Speed Run:** Return to the start and execute a high-speed run through the optimal path.

Separation of Features

- **Essential Features (MVP):**

- Maintain accurate digital (X , Y) coordinates and heading in memory.
- Autonomously map the maze and find the center in a single pass.
- Execute stable continuous PID motor control to navigate without crashing.

- **Optional Features (Stretch Goals):**

- **The Speed Run:** Return to the start and execute a high-speed run through the optimal path.
- **Live Wi-Fi Telemetry:** Broadcast the internal maze map in real-time to a laptop dashboard.

Technical and Hardware-Related Challenges

- Wheel Slip & Trajectory Drift:

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.
 - Solution: Highly tuned PID control using wheel encoders to constantly correct drift.

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.
 - Solution: Highly tuned PID control using wheel encoders to constantly correct drift.
- **I2C Address Conflicts:**

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.
 - Solution: Highly tuned PID control using wheel encoders to constantly correct drift.
- **I2C Address Conflicts:**
 - All three VL53L0X sensors share the same default I2C address.

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.
 - Solution: Highly tuned PID control using wheel encoders to constantly correct drift.
- **I2C Address Conflicts:**
 - All three VL53L0X sensors share the same default I2C address.
 - Solution: Sequentially wake sensors via XSHUT pins and dynamically assign unique addresses.

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.
 - Solution: Highly tuned PID control using wheel encoders to constantly correct drift.
- **I2C Address Conflicts:**
 - All three VL53L0X sensors share the same default I2C address.
 - Solution: Sequentially wake sensors via XSHUT pins and dynamically assign unique addresses.
- **ATmega32 Memory Limits (2KB SRAM):**

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.
 - Solution: Highly tuned PID control using wheel encoders to constantly correct drift.
- **I2C Address Conflicts:**
 - All three VL53L0X sensors share the same default I2C address.
 - Solution: Sequentially wake sensors via XSHUT pins and dynamically assign unique addresses.
- **ATmega32 Memory Limits (2KB SRAM):**
 - High risk of exhausting memory when storing maze states and processing interrupts.

Technical and Hardware-Related Challenges

- **Wheel Slip & Trajectory Drift:**
 - Physical motors never spin at perfectly identical speeds.
 - Solution: Highly tuned PID control using wheel encoders to constantly correct drift.
- **I2C Address Conflicts:**
 - All three VL53L0X sensors share the same default I2C address.
 - Solution: Sequentially wake sensors via XSHUT pins and dynamically assign unique addresses.
- **ATmega32 Memory Limits (2KB SRAM):**
 - High risk of exhausting memory when storing maze states and processing interrupts.
 - Solution: Use efficient bitwise operations and offload Flood Fill logic to the ESP32.

Future Scope and Improvements

- Live Wi-Fi Telemetry (Dashboard):

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.
 - Visualize robot coordinates and walls in real-time on a React/Node.js frontend.

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.
 - Visualize robot coordinates and walls in real-time on a React/Node.js frontend.
- **The Optimal Speed Run:**

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.
 - Visualize robot coordinates and walls in real-time on a React/Node.js frontend.
- **The Optimal Speed Run:**
 - Expand the software to a complete multi-pass solver.

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.
 - Visualize robot coordinates and walls in real-time on a React/Node.js frontend.
- **The Optimal Speed Run:**
 - Expand the software to a complete multi-pass solver.
 - After mapping, return to start and execute a high-speed run through the calculated shortest route.

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.
 - Visualize robot coordinates and walls in real-time on a React/Node.js frontend.
- **The Optimal Speed Run:**
 - Expand the software to a complete multi-pass solver.
 - After mapping, return to start and execute a high-speed run through the calculated shortest route.
- **Sensor Fusion (IMU Integration):**

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.
 - Visualize robot coordinates and walls in real-time on a React/Node.js frontend.
- **The Optimal Speed Run:**
 - Expand the software to a complete multi-pass solver.
 - After mapping, return to start and execute a high-speed run through the calculated shortest route.
- **Sensor Fusion (IMU Integration):**
 - Integrate an MPU6050 (Accelerometer & Gyroscope).

Future Scope and Improvements

- **Live Wi-Fi Telemetry (Dashboard):**
 - Broadcast the ESP32's internal maze map over Wi-Fi.
 - Visualize robot coordinates and walls in real-time on a React/Node.js frontend.
- **The Optimal Speed Run:**
 - Expand the software to a complete multi-pass solver.
 - After mapping, return to start and execute a high-speed run through the calculated shortest route.
- **Sensor Fusion (IMU Integration):**
 - Integrate an MPU6050 (Accelerometer & Gyroscope).
 - Fuse gyro data with wheel encoders for flawless straight headings and exactly 90° turns.